SSH, Firewall and SELinux Assignment

Task 1: Configure SSH for key-based authentication only

Step 1: Check the SSH service

Open your Linux terminal and switch to root:

su -

Check whether OpenSSH Server is installed:

rpm -q openssh-server

If it isn't installed, run:

dnf install openssh-server -y

Start and enable the service:

systemctl enable --now sshd

Verify:

systemctl status sshd

Look for active (running).

Step 2: Generate an SSH key pair

Switch to your normal user account. For example:

su - diya

Generate a key:

ssh-keygen -t ed25519

Press Enter to accept the default file location. Set a passphrase if you want additional protection.

The keys are saved in:

- ~/.ssh/id_ed25519 — private key; never share it.
- ~/.ssh/id_ed25519.pub — public key.

Step 3: Install your public key

For a local lab, add the public key to your account's authorized keys:

mkdir -p ~/.ssh

chmod 700 ~/.ssh

cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys

Test key authentication:

ssh -o PreferredAuthentications=publickey -o PasswordAuthentication=no diya@localhost

Replace diya with your actual username.

You should be able to log in using your key. If you set a key passphrase, you may be prompted for that passphrase.

Step 4: Disable password authentication

Switch to root:

su -

Open the SSH configuration:

vi /etc/ssh/sshd_config

Find or add these active settings:

PubkeyAuthentication yes

PasswordAuthentication no

In vi, press i to edit, then press Esc and type :wq to save.

Validate the configuration:

sshd -t

If there is no output, the syntax is valid. Restart SSH:

systemctl restart sshd

Verify the effective settings:

sshd -T | grep -E '^(pubkeyauthentication|passwordauthentication) '

Expected output:

pubkeyauthentication yes

passwordauthentication no

Finally, test key login again from a separate terminal. Do not close your existing session until it works.

Task 2: Configure the firewall using firewalld

RHEL 9 normally uses firewalld. We'll allow incoming SSH on port 22 and HTTP on port 80.

Step 1: Start the firewall

systemctl enable --now firewalld

Check its status:

firewall-cmd --state

Expected output:

running

Step 2: Check the active firewall zone

firewall-cmd --get-active-zones

For the commands below, I'll use the public zone. If your network interface is assigned to a different zone, use that zone instead.

Inspect the current rules:

firewall-cmd --zone=public --list-all

Step 3: Allow SSH and HTTP

Run:

firewall-cmd --permanent --zone=public --add-service=ssh

firewall-cmd --permanent --zone=public --add-service=http

These services allow TCP ports 22 and 80, respectively.

Remove other services listed in the zone, leaving only ssh and http. For example, if cockpit or dhcpv6-client is listed and should not be allowed, remove it:

firewall-cmd --permanent --zone=public --remove-service=cockpit

firewall-cmd --permanent --zone=public --remove-service=dhcpv6-client

Also remove any extra ports or rich rules shown by --list-all, while preserving any rules needed for your lab network. Then reload:

firewall-cmd --reload

Verify:

firewall-cmd --zone=public --list-all

The services line should contain only:

services: http ssh

Check the ports and rich rules lines too. They should not allow additional incoming services or ports. Note that other zones and IPv6 rules also need to be considered if they apply to your server.

Step 4: Verify listening ports

On the Linux server, run:

ss -tulnp

This displays listening sockets. It does not by itself prove which ports are reachable through the firewall.

If nmap is installed on another machine on the same network, scan your server's actual IP address:

nmap -Pn -p 1-1024 SERVER_IP

Replace SERVER_IP with the server's IP address. For this assignment, ports 22 and 80 should be reachable if their services are running, and other scanned ports should be filtered or closed. Remember that nmap results describe what is reachable from that particular scanning machine.

Task 3: Create automate_backup.sh

This script will create a timestamped compressed backup of /home inside /backups.

Step 1: Create the backup directory

As root:

mkdir -p /backups

chmod 700 /backups

Step 2: Create the script

vi /usr/local/bin/automate_backup.sh

Paste this script:

```
#!/bin/bash

set -o pipefail

BACKUP_DIR="/backups"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="$BACKUP_DIR/home_backup_$TIMESTAMP.tar.gz"

mkdir -p "$BACKUP_DIR" || exit 1

echo "Starting /home backup at $(date)"

if tar -czpf "$BACKUP_FILE" /home; then
    echo "SUCCESS: Backup created at $BACKUP_FILE"
else
    echo "ERROR: Backup failed"
```

```
    rm -f "$BACKUP_FILE"
    exit 1
fi


echo "Backup completed at $(date)"
```

Save and exit.

Step 3: Make it executable

```
chmod +x /usr/local/bin/automate_backup.sh
```

Step 4: Run and verify the backup

```
/usr/local/bin/automate_backup.sh
```

List the generated backup:

```
ls -lh /backups
```

Check the archive contents:

```
tar -tzf /backups/home_backup_YYYY-MM-DD_HH-MM-SS.tar.gz | head
```

Replace the example timestamp in the filename with the actual one created by your script.

Task 4: Check and enable SELinux Enforcing mode

Step 1: Check the current mode

```
getenforce
sestatus
```

Possible modes are Enforcing, Permissive, and Disabled.

Step 2: Switch to Enforcing mode if needed

If the current mode is Permissive, run:

```
setenforce 1
```

If it is already Enforcing, no change is needed.

Verify:

```
getenforce
```

Expected output:

```
Enforcing
```

If the result is Disabled, setenforce cannot enable SELinux at runtime. Do not modify boot configuration or reboot without checking your lab requirements first.

Task 5: Linux server security checklist

Use this checklist to compare the intended best practices with your actual server configuration.

After testing each item, select the status that matches your server. Don't mark an item complete until you've verified it.

1. SSH key-based authentication

Public-key login works and password authentication is disabled.

Verified / configured

Missing / needs improvement

Not checked yet

2. Restrictive firewall

Only required incoming services are allowed; for this lab, SSH and HTTP.

Verified / configured

Missing / needs improvement

Not checked yet

3. Regular backups

Timestamped backups are created and periodically checked for successful restoration.

Verified / configured

Missing / needs improvement

Not checked yet

4. SELinux enforcing mode

SELinux is Enforcing and file contexts are appropriate.

Verified / configured

Missing / needs improvement

Not checked yet

5. System updates

Security updates are applied regularly using trusted repositories.

Verified / configured

Missing / needs improvement

Not checked yet

6. Least privilege

Use normal accounts for routine work and sudo only when needed.

Verified / configured

Missing / needs improvement

Not checked yet

7. Log monitoring

Review SSH, firewall, authentication, and SELinux logs for suspicious activity.

Verified / configured

Missing / needs improvement

Not checked yet

8. Secure web services

Keep services updated and use HTTPS when handling real user traffic.

Verified / configured

Missing / needs improvement

Not checked yet